

Advanced NLP Assignment 2: Mixture-of-Experts Architectures, Custom Optimizers, and Decoding Strategies

Surya Manoj Pentakota
IIIT Hyderabad
manoj.pentakota@research.iiit.ac.in

Abstract

We report on three studies on a small decoder-only Transformers model pretrained with under a $\sim 30\text{M}$ -token budget, which is roughly $0.1\times$ the Chinchilla. First, we compare a dense feed-forward network against four sparse mixture-of-experts (MoE) layouts matched for stored parameters, with half the active FNN parameters, top-2 and shared-expert top-1 routing beat the dense baseline by about +2.1 BLEU (+10%) on vi $\rightarrow$ en and ja $\rightarrow$ en translation despite a modestly worse perplexity. Expert usage is balanced, with no observed language specification on this scale. Second, we implement AdamW, NadamW, Lion, and Muon from scratch on a single epoch English-language budget. Muon finishes 0.43 nats ($\approx 1.5\times$ perplexity) below AdamW, Lion is second, and NadamW gives a small but consistent lift over AdamW, mirroring the results from reference paper; but continuation BLEU dissociates from perplexity. Third, we study greedy, top- k , top- p , and beam search (widths 1/2/4) built strictly on plain `forward` calls with Pythia-160M on ROCStories. Mode-seeking decoders hug the model’s own distribution (self-ppl 1.8–2.0) yet never output the gold first token, which sampling matches in $\sim 1\%$ of rows; beam time is overhead-bound, not width-bound.

1 Introduction

This assignment studies three ingredients of modern model pretraining pipelines using one shared experiment frame: a small decoder-only Transformer (Vaswani et al., 2017) trained on a fixed token budget, with every comparison held to identical schedules and step counts. Part 1 asks whether sparse mixture-of-experts (MoE) feed-forward layers (Shazeer et al., 2017; Lepikhin et al., 2021; Fedus et al., 2022; Jiang et al., 2024) can trade stored parameters for active efficiency on a multilingual translation task. Part 2 re-implements four optimizers — AdamW, NadamW, Lion, Muon — from

the survey of Wen et al. (2025) and tests which update rule extracts the most from the data in a data-poor regime. Part 3 implements five decoder strategies on top of a pretrained language model and characterizes their behavior.

Following the assignment requirements, translation quality is measured with corpus BLEU (Papineni et al., 2002) and perplexity; the repeated gap between these two metrics is itself a finding in both Part 1 and 2. The compute regime is minimal: the budget is $\approx 0.1\times$ Chinchilla – compute-optimal token count of Hoffmann et al. (2022). The three parts are covered in Sections 2–4, with a summary in Section 5.

2 Part 1: Sparse Mixture-of-Experts Variants

2.1 Variants and parameter matching

We replace the feed-forward network (FFN) of a decoder-only Transformer with five variants – a generic dense FFN and other four following the sparse-gating blueprint of Shazeer et al. (2017) refined in Lepikhin et al. (2021); Fedus et al. (2022); Jiang et al. (2024): a softmax router over expert logits selects the top- k experts per token, combined by router weights (top-2) or taken alone (top-1); variant 4 keeps one always-on “shared” expert, guaranteeing every token a fixed path.

The parameter count matching is the fairness contract of the experiment (Table 1): variants 1–4 stores the *same* total FFN parameters, the dense FFN divided into quarter width experts; variant 5 stores $2\times$ the dense parameters so that its *active* parameter count matches variant 1’s *active* parameter count. Active FFN parameters thus range from 8.9% (v2) to 26.2% (v5) of the model total parameters. The router gate adds $d_{\text{model}} \times n_{\text{routed}}$ parameters per layer ($\approx 0.1\%$ of the FFN total), which is the reason for the slight difference between the v1–v4 totals.

Variant	Structure	Stored FFN	Active FFN	Active %
v1	Dense MLP, d_{ff} 1536	26,653,440	9,437,184	35.4
v2	4 experts $\times$ 384, top-1	26,665,728	2,371,584	8.9
v3	4 experts $\times$ 384, top-2	26,665,728	4,730,880	17.7
v4	3 routed $\times$ 384 + 1 shared, top-1	26,662,656	4,727,808	17.7
v5	4 experts $\times$ 768, top-2	36,102,912	9,449,472	26.2

Table 1: FFN parameter census (per-variant FFN + router gate totals; the shared $32k \times 384$ tied embedding, 12.3M, is excluded). Active counts cover the FFN only; attention and embeddings are identical across variants.

2.2 Experimental setup

All five variants train on [belumind/en-vi-ja-curated-500k-triplets](#), cast as two translation tasks per row (vi $\rightarrow$ en, ja $\rightarrow$ en) distinguished by a task-masked prompt. Text is encoded with byte-level BPE tokenizer of 32k vocab ([Sennrich et al., 2016](#)). The model is a decoder-only Transformer with (d_{model} 384, 8 layers, 6 heads, d_{ff} 1536, tied embeddings, dropout 0). Each variant sees the identical stream of 29,958,441 tokens under AdamW lr 8×10^{-4} , betas (0.9, 0.98), wd 0.01, clip 1.0, a 1.5M token warmup plus a cosine decay to 10%, fp16 AMP. Validation runs every 3M tokens (0.1x total tokens budget) for 10 checkpoints — a budget and schedule otherwise identical across variants.

2.3 Results

Table 2 summarizes the best-checkpoint results which it is observed to be the last checkpoint with complete training budget in all runs. Sparse routing beats the dense baseline on translation quality at half the active FFN parameters: v3 (top-2) and v4 (shared + top-1) post 23.45 and 23.47 BLEU against 21.34 for v1, a $\approx +2.1$ point (+10%) gain, and they also v5, which activates as many parameters as the dense model. All curves are smooth and monotone from 3M to 30M tokens, with no overfitting observed (train $\approx$ val loss within ~ 0.001 ; test/ppl only 0.04–0.09 above val).

Three patterns stand out from the results. First, **perplexity and BLEU dissociate**: v1 and v5 win on ppl (11.88/11.91) yet lose on BLEU (21.34/22.06) to v3/v4 (12.45/12.59 ppl, 23.45/23.47 BLEU): token-level CE does not track translation quality under this minimal training budget. Second, **ja $\rightarrow$ en is the differentiator**. vi $\rightarrow$ en BLEU is tight across variants (25.07–27.75), but the sparse winners lift ja $\rightarrow$ en by +1.6 to +2.4 points over v1, while v5 *hurts* it (−0.8): narrow quarter-width experts generalize better to the harder language than wide half-width experts. Third, **v2 is a**

pure compute story: a quarter of the active FFN costs only −0.34 BLEU (−1.6%) while perplexity degrades 27% suggesting sparsity at fixed capacity loses some little downstream quality.

Degeneracy (dense only). Greedy decoding of v1 emits an *empty* translation on 80/600 rows (13.3%; 48 vi, 32 ja), while every MoE variant stays below 1% (with highest at 5/600 rows in v4). Our hypothesis which is just an interpretation and not a proven mechanism is that the first target token is never directly supervised, its label sits inside the masked task prefix, leaving the representation at the source-final EOS position underdetermined; the dense FFN hence drifts toward an “idle $\rightarrow$ EOS” interpretation, while MoE routing keeps every position on a live expert manifold. These are standard NMT degeneracies with fixes such as first-step EOS penalties, length penalties, and beam search which are what Part 3 of the assignment studies.

Expert usage. Figure 1 shows per-expert usage over the test split. Load balancer is healthy everywhere: relative to uniform assignment (25% per expert and 33% for v4’s three routed experts), the busiest expert carries $1.10\times$ (v2), $1.30\times$ (v3), $1.33\times$ (v4), and $1.58\times$ (v5) of their share, with no expert starved. Winner-take-more is not observed and is mild and grows with expert width (v3 $\rightarrow$ v5), in line with the capacity-collapse tendency analysed by [Chi et al. \(2022\)](#). The surprising results observed is that **language specialization is none**. Per-expert vi/ja ratios match the corpus-wide 1.108 in every variant with a max deviation 1.24, which infers both languages are routed identically. As [Jiang et al. \(2024\)](#) found for topical assignment, no routing pattern emerges at four experts and 30M tokens, clusters only capture generic token statistics and not language identity; v4’s shared expert is just workhorse rather than a language expert which we can observe as metrics are consistent with v4 $\approx$ v3 in quality.

Variant	train/loss	val/ppl	test/ppl	BLEU (600)	BLEU vi→en	BLEU ja→en	empty preds
v1 dense	2.477	11.88	11.93	21.34	25.07	17.40	80 / 600
v2 top-1	2.644	15.05	15.14	21.00	25.10	16.76	2 / 600
v3 top-2	2.535	12.38	12.45	23.45	27.75	19.01	3 / 600
v4 shared+top-1	2.484	12.55	12.59	23.47	27.06	19.75	5 / 600
v5 top-2 wide	2.452	11.91	11.96	22.06	27.40	16.63	1 / 600

Table 2: Best-checkpoint results per variant: sacrebleu corpus BLEU over 600 greedy translations (300 test rows $\times$ 2 languages); empty predictions are included (matches the logged corpus BLEU). Best values per column in bold.

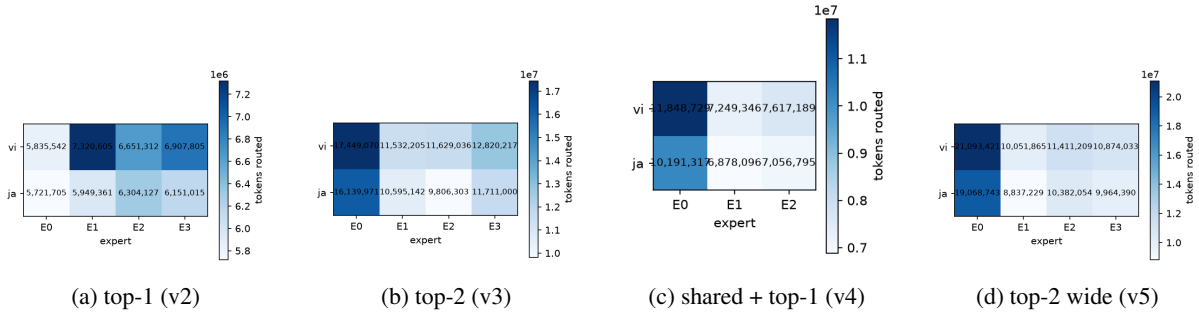


Figure 1: Per-expert usage heatmaps (layers $\times$ experts) from the best checkpoints. Counts include padded positions in each batch, inflating absolute values by $\sim 10\text{--}20\%$ systematically across variants; relative patterns are unaffected.

2.4 Discussion

The Part 1 takeaway is that top-2 sparsity is worth its complexity at this scale: two half-width experts beat one full-width FFN on BLEU with half the active parameters, and the advantage concentrates on the harder language direction. The shared-expert design of v4 buys little beyond a small ja→en edge, and widening experts (v5) is not worth even when active counts are matched as width trades away the per-expert specialization that routing needs. Two caveats temper these conclusions: the 30M-token budget is small enough that BLEU differences of 1–2 points carry substantial noise, and we deliberately used greedy decoding, which exposes v1’s empty-output failure mode; a length-penalized or beam decoder would change the absolute numbers, though not the relative ordering. The absence of language specialization is a genuinely negative result worth flagging: at four experts, routing just learns the token statistics rather than language identity, and any claim of “language experts” would need larger models and substantially more data.

3 Part 2: Custom Optimizers

3.1 The four optimizers

Following the survey’s Table 1 (pseudocode in its Appendix A) (Wen et al., 2025), we implement four optimizers from scratch, all subclassing `torch.optim.Optimizer` directly.

AdamW (Kingma and Ba, 2015; Loshchilov and Hutter, 2019) is the baseline: bias-corrected first and second moments plus *decoupled* weight decay applied outside the adaptive step, the fix that cleanly separates lr and wd tuning (Loshchilov and Hutter, 2019). State: $2 \times (m, v)$.

NadamW (Dozat, 2016) swaps in Nesterov momentum: the update uses the look-ahead momentum estimate including the current gradient. State: $2 \times$.

Lion (Chen et al., 2023) applies a sign operation to a momentum interpolation: uniform-magnitude updates that regularize through update noise, at $1 \times$ state; it typically wants a smaller lr and a larger wd than Adam.

Muon (Jordan et al., 2024) treats each 2-D weight as a matrix: a few Newton–Schulz iterations approximate the orthogonal polar factor of the momentum, so the update is a matrix (spectral-norm) preconditioner rather than an entry-wise scaling; embeddings, head, and biases stay on an AdamW branch. State: $\approx 1.31 \times$. Table 3 lists the locked recipe.

3.2 Experimental setup

All four optimizers are trained on the English side of [browndw/human-ai-parallel-corpus](#) with a byte-level BPE tokenizer of 16k vocabulary (Sennrich et al., 2016), split doc-grouped 90/5/5 so that validation and test

Optimizer	Category	lr	wd	state
AdamW	Baseline	5e-4	0.1	2.00×
NadamW	Variance-reduced	5e-4	0.1	2.00×
Lion	Memory-efficient	1.5e-4	0.6	1.00×
Muon	Matrix-based	NS 0.03 / Adam 3e-4	0.1	≈1.31×

Table 3: Locked hyperparameter recipe. All runs: betas (0.9, 0.95) on the Adam-family branches (Muon momentum 0.95); identical stream/budget/steps. Anchors: the survey’s tuned AdamW wd ≈ 0.1 ; Lion lr $\approx$ AdamW/3 with its tuned wd 0.6 (Chen et al., 2023); Muon’s NS branch at 0.03 following the survey’s shrinking-model trend (Wen et al., 2025; Jordan et al., 2024).

see unseen documents (the test split provides 415 human chunk1→chunk2 pairs for BLEU). The model is the Part-1 dense configuration (20,509,440 total / 14,168,832 non-embedding parameters at 16k vocab). The budget is exactly one epoch: 29,511,235 tokens ($\approx 0.1 \times$ Chinchilla), 1,866 steps of batch 32×512 , with a 5% per-group warmup and cosine decay to 10%, seed 42, and an identical shuffle stream for all four runs. We train in bf16 autocast with fp32 master parameters and a global gradient clip of 1.0, and validate every 2,951,123 tokens (10 checkpoints).

A methodology note is in order: an earlier fp16 run of exactly this recipe collapsed with training loss rose from step 100 onward ($7.50 \rightarrow 9.13$ by 4.75M tokens, val ppl 2,696). The failure pattern across the whole project was always fp16 without a loss scaler, so my understanding from the observations is that fp16 *censors* the small-gradient tail (underflow to zero plus absorption in fp16 accumulation), a systematic one-directional error that accumulates $\propto \text{lr} \cdot t$, that weight decay cannot cancel, and that lives in the rare-token rows of the tied head. bf16 keeps fp32’s exponent range, so its errors are unbiased rounding that averages out and it needs no scaler. Also inline with previous understanding Part 1 survived fp16 because its translation gradients are large; Part 2’s next-token task could not.

3.3 Results

Table 4 and Figure 2 give the end-of-budget metrics. The headline is **Muon finishes 0.43 nats below AdamW** (4.1452 vs. 4.5751, i.e. $\approx 1.5 \times$ lower perplexity: 63.13 vs. 97.04). The dynamics are instructive, Muon starts *behind* the Adam family (351.1 vs. 337.2 ppl at the first checkpoint, because its embeddings train at 3e-4 against the family’s 5e-4), crosses by the second checkpoint, then pulls away monotonically meaning matrix preconditioning compounds over the run. This reproduces the

survey’s small-scale headline that matrix-based optimizers dominate (Wen et al., 2025).

Lion is the “often tracks AdamW, sometimes better” case, and here it is the better case: second place at 4.4425 (0.13 nats under AdamW), with the slowest start (450.7 ppl) and the steepest subsequent descent among the scalar family, at half the optimizer state (1.00×). **NadamW** delivers a small but clean lift over AdamW (+0.054 nats), which is consistent with the survey’s finding that variance-reduced variants beat vanilla AdamW modestly. All four curves are monotone in all 10 checkpoints, and the final ordering (Muon $\gg$ Lion $\geq$ NadamW $\geq$ AdamW) is stable from roughly the third checkpoint onward with no mid-run crossings.

The BLEU story is different and deliberately reported as such: **Muon has the weakest final continuation BLEU (0.573) while leading every perplexity checkpoint**, and NadamW/Lion lead BLEU (1.069/1.040) with mid-tier perplexity. As in Part 1, token-level CE and lexical match dissociate; with 415 pairs and a single reference at this scale, each BLEU point is estimator-noisy in a $\sim 0.5\text{--}1.1$ band, so we treat val loss as the primary signal and the dissociation itself as the finding.

3.4 Discussion

The regime explains the results. A 14.2M non-embedding model is compute-optimal at $\approx 283\text{M}$ tokens (1x Chinchilla == 20x non-embedding parameters) (Hoffmann et al., 2022); our 29.5M-token budget is $\approx 0.10 \times$ that, the extreme low data to model regime in which Wen et al. (2025) find matrix-based optimizers most advantageous and in which their claimed speedups are largest (decreasing from $1.4 \times$ at 0.1B to $1.1 \times$ at 1.2B parameters). Muon’s 0.43-nat lead here is regime-consistent, and its scaling story has since been confirmed at 16B scale by Xu et al. (2025). The survey’s second methodological warning also played out literally in debugging that intermediate-checkpoint compar-

Optimizer	val ppl	val loss	train loss	BLEU fin.	BLEU best
Muon	63.13	4.1452	4.2985	0.573	0.803
Lion	84.99	4.4425	4.5478	1.040	1.131
NadamW	91.93	4.5211	4.6281	1.069	1.069
AdamW	97.04	4.5751	4.6851	0.814	0.905

Table 4: End-of-budget metrics (29,523,280 tokens, 1,866 steps). BLEU = test split (415 human chunk1→chunk2 pairs, single reference, 128-token greedy continuations); *fin.* = final checkpoint, *best* = any checkpoint. Runtimes: 2,277–2,538 s per run.

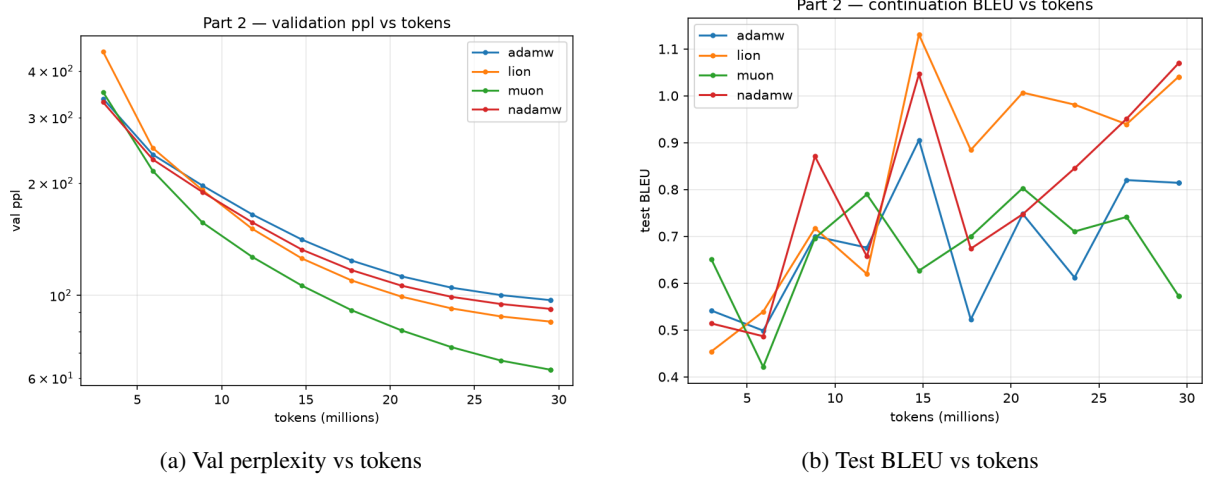


Figure 2: Curves over the token budget (10 checkpoints per run).

isions mislead, our compressed $0.1\times$ calibration runs looked “slightly hot at worst,” while the real disease (sustained-lr drift from the fp16 gradient bias) only surfaced at full-length schedules. Finally, the small single-reference BLEU metric is a coarse instrument at this scale (Papineni et al., 2002); ranking four optimizers by a ± 0.2 -point band of noisy scores would overinterpret the data. BLEU here buys one honest claim that optimizers can lead on every ppl checkpoint and still trail on lexical match.

4 Part 3: Decoding Strategies

4.1 Strategies

We implement wight decoding configurations: greedy; top- k sampling with $k=10, 50$ (Fan et al., 2018); top- p sampling with $p=0.8, 0.95$ (Holtzman et al., 2020); and beam search with widths 1,2,4 (Freitag and Al-Onaizan, 2017); all built strictly on `model.forward()`. Greedy takes the argmax of the last-row logits; top- k and top- p truncates and renormalizes the next token distribution and then sample; beam search keeps the W highest scoring hypothesis per step by cumulative log-probability, moves EOS-terminated hypothesis to a finished list and returns the best finished or live hypothesis

scored by *raw* (unnormalized) cumulative log-prob.

4.2 Evaluation protocol

For evaluations Prompts and continuations come from ROCStories (Mostafazadeh et al., 2016) with text split of 1,000 stories subsampled with seed (42) where prompt = sentence 1, gold = the remaining four. The generator model is EleutherAI Pythia-160M (Biderman et al., 2023) in fp16, wrapped so that every strategy sees one forward contract; generation runs to *max_new* = 96 tokens with prompts truncated to 256. Metrics used are *gold ppl* (29.52), the model’s NLL of the true continuations which is a strategy-independent calibration of how hard the task is; *self ppl*, the NLL of each config’s own output (low = strategy stays in the model’s high-likelihood region, which can be quality or degeneration); *first-token accuracy*, the share of rows whose first generated token equals the gold’s first token; mean generated length and the empty fraction; and wall-clock decode time.

4.3 Results

Table 5 collects the results. findings:

1. Self-ppl ladder: mode-seeking $\ll$ sampling. Greedy (1.99) and beam-1 (2.00) hug the model’s

own distribution with wider beams pushing it lower (1.82, 1.77 at widths 2,4 meaning beam search is doing its job of finding higher-likelihood continuations). Sampling self-ppl inflates with the sampled distribution’s entropy 7.9 (top- k -10) $\rightarrow$ 15.3 (top- k -50) $\rightarrow$ 17.3 (top- p -0.8) $\rightarrow$ 45.8 (top- p -0.95). Broader support means more mass on the low-probability tokens, hence worse self-NLL.

2. The model prefers its own text. Self-ppl (1.8–46) is far below gold ppl (29.5) for the deterministic configs the model’s own continuations are $\sim 15\text{--}17\times$ more likely per token than the gold ones, the classic self-preference/degeneration signature mentioned in [Holtzman et al. \(2020\)](#); [Welleck et al. \(2020\)](#), which is also visible qualitatively in generated rows that loop phrases (e.g. “I was a little bit of a loner, and I was a little bit of a loner, ...”). Low self-ppl is not a quality score against the reference and the report’s metrics deliberately keep gold-relative signals separate.

3. First-token accuracy: zero for deterministic decoders. Greedy and all beams match the gold first token in 0/1,000 rows, while sampling lands on it in 0.8–1.8% of rows. At this scale the gold first token carries only $\sim 1\text{--}2\%$ probability mass and is rarely the mode and sampling from a truncated distribution (narrower sets more so as top- k -10 at 1.8% beats top- k -50 at 1.1%) occasionally *lucks into* it to get the gold first token. These are small counts (≤ 18 hits), so the ordering is suggestive and the zero-vs-nonzero pattern is the point which differentiates the sampling methods.

4. Beam-1 vs. greedy: identical by construction, fp16 rounding apart. Algorithmically, width-1 beam is greedy. It is observed 518/1,000 rows are token-identical and the other 482 diverge on some near-tie and never re-merge. The cause for this deviation is not logical implementation difference and it is (a) fp16 path-dependent rounding where greedy runs batched (32 rows, padded) and beam row-by-row, so cuBLAS reduction orders differ and logits differ at fp16 round-off levels, and (b) the selection operation $\text{argmax}(\text{logits})$ vs. $\text{topk}(\text{cum} + \text{log_softmax}(\text{logits}))$ are monotone-equivalent in exact math but the extra log_softmax rounding can flip near-ties. Since argmax is discontinuous, one ulp-level difference anywhere along 96 steps suffices. With 48.2% of rows differing, the implied per-step flip rate is $\approx 0.7\%$, which is consistent with rare fp16 near-ties

and hence not a bug.

5. Beam width: better self-ppl, earlier stops.

Mean length shrinks $96.0 \rightarrow 93.9 \rightarrow 90.3$ from beam-1 to beam-4 meaning wider beams find EOS-bearing hypotheses that beat continuing on raw cumulative log-prob which is the classic unnormalized length bias ([Murray and Chiang, 2018](#)).

6. Timing is overhead-bound, not width-bound in Beam search.

Beam runs $\approx 11.6\times$ slower than greedy (1,322/1,341/1,339 s vs. 114.4 s) and is *flat* across widths. The FLOP model $O(\max_{new} \cdot T^2 \cdot W)$ predicts width scaling and the wall clock does not show it because beam’s per-row Python loop issues $\sim 96k$ tiny batch-1 forwards (vs. $\sim 3k$ batch-32 forwards for the batched configs) meaning the run is Python-overhead bound at these sizes. We state the complexity model analytically and note the wall clock measures overhead and not FLOPs. A cross-row batched beam would reveal the true scaling.

4.4 Discussion

Part 3’s decoders are separated into two families: deterministic search, which stays inside the model’s own high-likelihood region and never touches the gold first token and the seeded sampling which occasionally does but pays for this in self-NLL. Neither family can be claimed to be “better” in abstract as self-ppl cannot rank them against the reference ([Holtzman et al., 2020](#)), here the deliverable is the categorization of the families of approaches rather than ranking them. The beam-1 $\equiv$ greedy equivalence analysis is a reminder that the fp16 path dependence is a real reproduction consideration at the token level meaning bit-identical outputs are not guaranteed (this claim needs a bit more study and reproduction with fp32 to confidently claim it), and the flat beam timing curve is a reminder to separate the FLOP-scaling claims from the overhead measurements. The connection back to Part 1 is direct here with our understanding with Part 3 that deterministic decoding with explicit EOS handling and length-aware scoring is precisely the standard machinery for fixing the v1’s empty translation failures and to make beam search affordable in production we can replicate the buffered KV cache implementation which we used for Part 2’s decoding step in the evaluations.

Config	self ppl	1st-tok acc	mean len	empty	time (s)
greedy	1.99	0.000	96.0	0.0%	114.4
top-k-10	7.87	0.018	94.3	0.0%	119.6
top-k-50	15.26	0.011	92.9	0.0%	119.6
top-p-0.8	17.29	0.008	92.2	0.1%	123.3
top-p-0.95	45.81	0.008	92.6	0.1%	123.7
beam-1	2.00	0.000	96.0	0.0%	1,321.6
beam-2	1.82	0.000	93.9	0.0%	1,341.4
beam-4	1.77	0.000	90.3	0.0%	1,338.7

Table 5: Part 3 results (1,000 stories, max_new 96, seed 42, Pythia-160M fp16). Gold ppl baseline = 29.52. First-token accuracy = share of rows whose first generated token equals the first gold token (EOS-as-first-token = miss).

5 Conclusion

Across three experiments on $\sim 30\text{M}$ -token budgets, the recurring result is that architectural and algorithmic choices dominate token-level metrics while reference-based metrics behave independently. Sparse top-2 MoE beats a dense FFN on translation BLEU (+10%) at half the active parameters, with balanced, language-agnostic routing; Muon beats AdamW by 0.43 nats in a data-poor regime where matrix preconditioners are known to help most and decoding strategy determines whether a model’s output stays in its own high-likelihood region (greedy/beam) or occasionally matches the reference (sampling). Each comparison was held to identical streams, budgets, and schedules, and each headline claim survives a check against its own artifact that the ppl/BLEU dissociation appears in Parts 1 and 2 alike, and the beam-1/greedy divergence traces to fp16 rounding rather than logic. At $0.1\times$ Chinchilla scale the absolute numbers are small, but the relative findings align with the reference literature on MoE capacity, optimizer speedups, and text degeneration. Models, runs are listed in Section 5.

Resources and Reproducibility

- Trained models (Part 1, variants v1–v5): <https://huggingface.co/Surya4/anlp2-part1>
- Trained models (Part 2, four optimizers): <https://huggingface.co/Surya4/anlp2-part2>
- Training runs / curves: <https://forge.coreweave.com/wandb/suryamanojphy31-iiit-hyderabad/anlp-assignment2>

- Datasets: <https://huggingface.co/datasets/belumind/en-vi-ja-curated-500k-triplets> and <https://huggingface.co/datasets/browndw/human-ai-parallel-corpus>

References

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, Usven Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. 2023. [Pythia: A suite for analyzing large language models across training and scaling](#). In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 2397–2430, Honolulu, HI, USA.
- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. 2023. [Symbolic discovery of optimization algorithms](#). In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, New Orleans, LA, USA.
- Zewen Chi, Li Dong, Shaohan Huang, Damai Dai, Shuming Ma, Barun Patra, Saksham Singhal, Payal Bajaj, Xia Song, Xian-Ling Mao, Heyan Huang, and Furu Wei. 2022. [On the representation collapse of sparse mixture of experts](#). In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, pages 34600–34613, New Orleans, LA, USA.
- Timothy Dozat. 2016. [Incorporating nesterov momentum into adam](#). In *International Conference on Learning Representations (ICLR), Workshop Track*, San Juan, Puerto Rico.
- Angela Fan, Mike Lewis, and Yann Dauphin. 2018. [Hierarchical neural story generation](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 889–898, Melbourne, Australia. Association for Computational Linguistics.

- William Fedus, Barret Zoph, and Noam Shazeer. 2022. [Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity](#). *Journal of Machine Learning Research*, 23(120):1–39.
- Markus Freitag and Yaser Al-Onaizan. 2017. [Beam search strategies for neural machine translation](#). In *Proceedings of the First Workshop on Neural Machine Translation (WMT17)*, pages 56–60, Vancouver, Canada. Association for Computational Linguistics.
- Jordan Hoffmann et al. 2022. [Training Compute-Optimal Large Language Models](#). *arXiv preprint arXiv:2203.15556*. ArXiv:2203.15556 [cs.CL].
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. [The curious case of neural text de-generation](#). In *International Conference on Learning Representations (ICLR)*, Addis Ababa, Ethiopia.
- Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, et al. 2024. [Mixtral of experts](#). *arXiv preprint arXiv:2401.04088*. ArXiv:2401.04088 [cs.LG].
- Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. 2024. [Muon: An optimizer for hidden layers in neural networks](#). Technical blog post.
- Diederik P. Kingma and Jimmy Ba. 2015. [Adam: A method for stochastic optimization](#). In *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA.
- Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. [GShard: Scaling giant models with conditional computation and automatic sharding](#). In *International Conference on Learning Representations (ICLR)*.
- Ilya Loshchilov and Frank Hutter. 2019. [Decoupled weight decay regularization](#). In *International Conference on Learning Representations (ICLR)*, New Orleans, LA, USA.
- Nasrin Mostafazadeh, Nathanael Chambers, Xiaodong He, Devi Parikh, Dhruv Batra, Lucy Vanderwende, Pushmeet Kohli, and James Allen. 2016. [A corpus and cloze evaluation for deeper understanding of commonsense stories](#). In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 839–849, San Diego, CA, USA. Association for Computational Linguistics.
- Kenton Murray and David Chiang. 2018. [Correcting length bias in neural machine translation](#). In *Proceedings of the Third Conference on Machine Translation (WMT18), Volume 1: Research Papers*, pages 212–223, Brussels, Belgium. Association for Computational Linguistics.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. [BLEU: a Method for Automatic Evaluation of Machine Translation](#). In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. [Neural machine translation of rare words with subword units](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. [Outrageously large neural networks: The sparsely-gated mixture-of-experts layer](#). In *International Conference on Learning Representations (ICLR)*, Toulon, France.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 5998–6008, Long Beach, CA, USA.
- Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. 2020. [Neural text generation with unlikelihood training](#). In *International Conference on Learning Representations (ICLR)*, Addis Ababa, Ethiopia.
- Kaiyue Wen, David Hall, Tengyu Ma, and Percy Liang. 2025. [Fantastic Pretraining Optimizers and Where to Find Them](#). *arXiv preprint arXiv:2509.02046*. ArXiv:2509.02046 [cs.LG].
- Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. 2025. [Muon is scalable for LLM training](#). *arXiv preprint arXiv:2502.16982*. ArXiv:2502.16982 [cs.LG].